

Lab 5: Neural Networks

MLP Classifier for Handwritten Digit Recognition

Objective

In this lab, you will:

1. Load a classic **multi-class** dataset (digits) from `sklearn.datasets`.
2. **Preprocess** the data (e.g., feature scaling).
3. Build and train an **MLPClassifier** with different hyperparameters.
4. Evaluate the model using accuracy, confusion matrix, and classification report.
5. Tune hyperparameters using `GridSearchCV` or `RandomizedSearchCV`.

Task A1: Data Loading and Exploration

1. Import the digits dataset from scikit-learn using `load_digits()`.
2. Print out the shape of the data (features) and the target to understand how many samples and features there are.
3. Visualize a few digits.

Task A2: Train-Test Split & Scaling

1. Split your dataset into training and testing sets (e.g., 80% train, 20% test).
2. Apply feature scaling to your training data (e.g., `StandardScaler`) and then apply the same transformation to your test data.

Task A3: Build and Train an MLPClassifier

1. Create an **MLPClassifier** using scikit-learn.

Hint:

- ```
from sklearn.neural_network import MLPClassifier
```
2. Start with a single hidden layer (e.g., 64 neurons), `activation='relu'`, and `max_iter=300`.
  3. Fit the model on the **scaled** training data.
  4. Evaluate the model on both the training set and the test set to check for overfitting or underfitting.

### Task A4: Confusion Matrix & Classification Report

1. Plot the **confusion matrix** for your predictions on the test set.
2. Print the **classification report** to view precision, recall, and F1-score for each digit class (0–9).

### Task A5: Hyperparameter Tuning

1. Set up a **parameter grid** to search over different hidden layer sizes (e.g., (64,), (100,)), activation functions ('relu', 'tanh', or 'logistic'), alpha (regularization), and possibly different `learning_rate_init`.

#### Hint:

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {'activation': ['relu', 'tanh', 'logistic'],
 'hidden_layer_sizes': [(64,), (100,)]}
grid_search = ...
```

2. Use GridSearchCV (or RandomizedSearchCV) with cross-validation to find the best hyperparameters.
3. Compare the best model's performance to the default model.

**Submission Requirement:**

1. Complete in a Jupyter notebook file, name your file as " name student number#.ipynb" and submit it through "Lab 5 Programming Assignment" on iSpace.
2. Please provide necessary outputs to support your assignment. (e.g., printing necessary information or visualizing through graphs and tables).

Deadline: 11:59 pm, Mar.24, 2026. Late submission = 0; Plagiarism (cheating) = F.